

Understanding Selectors in Frontend Development and Cypress Testing

Selectors are a fundamental part of frontend development and testing. They enable developers and testers to interact with and manipulate elements within the Document Object Model (DOM). In the context of automated testing, particularly with tools like Cypress, selectors play a crucial role in ensuring that tests are robust, maintainable, and reliable. This blog delves into the intricacies of selectors, the challenges of traversing the DOM, and the importance of selector stability, with a special focus on the Cypress frontend testing framework.

What Are Selectors?

Selectors are patterns used to identify and interact with elements on a webpage. They can be simple, such as targeting an element by its ID or class name, or complex, using hierarchical structures and attributes. The primary types of selectors include:

1. **ID Selectors:** `#elementId`
2. **Class Selectors:** `.elementClass`
3. **Tag Selectors:** `div`, `span`,
4. **Attribute Selectors:** `[type="text"]`, `[href="/home"]`
5. **Data Attribute Selectors:** `[data-cy="button-confirm"]`
6. **Combinator Selectors:** `div > p`, `div + p`, `div ~ p`
7. **XPath Selectors:** `//div[@id='elementId']`

Selectors are crucial for both CSS styling and JavaScript manipulation. However, their role in automated testing frameworks like Cypress is where they truly shine.

The Role of Selectors in Cypress

Cypress is a popular end-to-end testing framework for web applications. It enables developers to write tests that interact with the application just like a real user would. These interactions are heavily reliant on selectors to identify the elements to be tested.

Basic Selector Usage in Cypress

Cypress provides a variety of ways to select elements. The simplest method is using the `cy.get()` command:

```
// Select an element by class
cy.get('.btn-submit').click();
```

Cypress also supports more complex selectors, combining multiple criteria to pinpoint elements accurately:

Cypress Handout - 3

```
// Select an element by attribute and text content
cy.get('button[type="submit"]').contains('Submit').click();
```

Traversing the DOM with Cypress

Traversing the DOM is a common task in testing, allowing you to move through the element hierarchy to interact with child, parent, sibling, or descendant elements. Cypress offers several commands to facilitate this:

- **.find()**: To find descendant elements
- **.parent()**: To get the immediate parent
- **.parents()**: To get all ancestors
- **.children()**: To get immediate children
- **.next()**: To get the next sibling
- **.prev()**: To get the previous sibling
- **.within()**: To get elements within a certain element

Traversing the DOM is essential for testing components and their interactions within complex nested structures. Below, we explore each traversal method in more detail:

.find()

The **.find()** command is used to locate descendant elements of the selected element. It is particularly useful when you want to narrow down your selection to a specific part of the DOM tree.

```
// Find the input field within a form
cy.get('form').find('input-field').type('Sample text');
```

In this example, the command starts by selecting the **form** element and then finds the **.input-field** within that form. This is ideal for cases where the input field may exist in multiple places, but you want to target the one within a specific form.

.parent()

The **.parent()** command retrieves the immediate parent of the selected element. This is useful when you need to move up one level in the DOM tree.

```
// Get the parent form of an input field
cy.get('input-field').parent().should('have.class', 'form-container');
```

This command ensures that the **.input-field** is contained within a parent element with the class **form-container**.

.parents()

The **.parents()** command retrieves all ancestor elements of the selected element, optionally filtered by a selector.



Cypress Handout - 3

```
// Find all ancestors of an input field with a specific class
cy.get('.input-field').parents().should('have.class', 'form-wrapper');
```

This method is useful when you need to verify that an element is within a particular section of the DOM, regardless of its immediate parent.

.children()

The **.children()** command gets the immediate children of the selected element, filtered by an optional selector.

```
// Get all child elements of a form
cy.get('form').children('.input-field').should('have.length', 3);
```

This example checks that a **form** element contains exactly three children with the class **input-field**.

.next() and .prev()

These commands navigate to the next or previous sibling element of the current element.

```
// Get the next sibling of a list item
cy.get('.list-item').next().should('have.class', 'list-item-next');

// Get the previous sibling of a list item
cy.get('.list-item').prev().should('have.class', 'list-item-prev');
```

These methods are particularly useful for validating the order of elements or for interacting with adjacent elements.

.within()

This function will limit the search for an additional element, within the boundaries of the first element. This would be the perfect option for finding elements that repeat frequently inside the DOM. In this example Cypress will only look for the **name** element inside the tree of **.form**.

```
cy.get('.form')
  .within(() => {
    cy.get('name').should('be.visible')
  })
```

Chaining Traversal Commands

Cypress allows chaining multiple traversal commands to create complex and precise selections. This is particularly useful when dealing with deeply nested DOM structures.



Cypress Handout - 3

```
// Chain commands to navigate through the DOM
cy.get('.nav')
  .find('ul')
  .children('li')
  .eq(2)
  .find('a')
  .should('have.attr', 'href', '/target-page');
```

In this example, the commands navigate from a navigation bar (**.nav**) to the third list item within a **ul** and then to an anchor tag within that list item, asserting that the **href** attribute is as expected.

It is considered bad practice to chain so many navigational commands. It should always be clear which element is being manipulated in the test. Using such exaggerated chaining makes it unclear which element is being manipulated.

Using XPath Selectors in Cypress

Although Cypress primarily supports CSS selectors, there are plugins available to enable XPath usage. XPath can be beneficial for selecting elements based on complex conditions or when dealing with XML data. To use XPath selectors in Cypress, you first need to install the **cypress-xpath** plugin:

```
npm install -D cypress-xpath
```

Then, include the plugin in your Cypress [support file](#):

```
require('cypress-xpath');
```

With this setup, you can now use XPath selectors in your Cypress tests:

```
// Select an element using XPath
cy.xpath('//button[@type="submit"]').click();
```

XPath expressions provide a powerful way to navigate and query the DOM, especially useful when CSS selectors are not sufficient. Always be careful when using xpath, it is generally known that xpath can introduce some amount of flakiness. Element selection such as **:nth-child(2)** are especially vulnerable for changes in the application. This could lead to false positives in your testing.

Ensuring Stability of Selectors

One of the key challenges in automated testing is maintaining the stability of selectors. Tests can become fragile if selectors are tightly coupled to the implementation details that frequently change. Here are some strategies to ensure selector stability.

Use Data Attributes

Data attributes (**data-***) are specifically designed to store extra information, which can be leveraged for selecting elements in tests without affecting styling or functionality.



Cypress Handout - 3

```
// HTML
<button data-cy="submit-btn">Submit</button>

// Cypress test
cy.get('[data-cy="submit-btn"]').click();
```

Avoid Overly Specific Selectors

While it's tempting to use very specific selectors to ensure you select the correct element, these can break easily if the DOM structure changes. Aim for a balance between specificity and robustness.

Use Semantic HTML

Using semantic HTML elements like `<nav>`, `<header>`, `<footer>`, etc., can help create more reliable and meaningful selectors.

Custom Commands

Create custom Cypress commands to encapsulate common selector patterns. This abstraction can help manage changes more efficiently.

```
Cypress.Commands.add('getSubmitButton', () => {
  return cy.get('[data-cy="submit-btn"]');
});

// Usage in test
cy.getSubmitButton().click();
```

Best Practices for Using Selectors in Cypress

To maximize the effectiveness and reliability of your Cypress tests, consider the following best practices.

Prioritize Readability and Maintainability

Write selectors that are easy to understand. Future you (or other developers) will appreciate it.

Leverage Cypress's Built-in Assertions

Cypress offers a range of assertions that can be used to verify element states and content. This reduces the need for complex selectors and makes tests more readable.

```
cy.get('[data-cy="submit-btn"]').should('be.visible').and('contain', 'Submit');
```

Use Aliases for Reusable Elements

If you need to interact with the same element multiple times, use aliases to avoid repeated selector logic. This will always reduce the amount of complexity in your testing framework.



Cypress Handout - 3

```
cy.get('[data-cy="submit-btn"]').as('submitButton');  
cy.get('@submitButton').click();  
cy.get('@submitButton').should('be.disabled');
```

Handle Dynamic Elements Gracefully

Web applications often have dynamic elements that appear or change based on user interactions. Use Cypress commands like `.should('exist')`, `.should('be.visible')`, and `.should('not.exist')` to handle these cases.

```
cy.get('[data-cy="loading-spinner"]').should('not.exist');  
cy.get('[data-cy="content"]').should('be.visible');
```

Traversing the DOM: Advanced Techniques

While basic traversal methods are useful, more advanced techniques can provide finer control and flexibility in your tests.

Chaining Commands

Cypress allows chaining of commands to create more complex test scripts, enabling you to interact with hard to locate elements in a fairly clear manner.

```
cy.get('form')  
  .find('input-field')  
  .type('Sample text')  
  .parents('form')  
  .submit();
```

Conditional Testing

Sometimes, you need to perform actions based on the presence or state of an element. Cypress's `.then()` method can help here.

```
cy.get('body').then(($body) => {  
  if ($body.find('[data-cy="modal"]').length > 0) {  
    cy.get('[data-cy="modal"]').should('be.visible').find('close-btn').click();  
  }  
});
```

Custom Assertions

Cypress's `should` and `expect` commands can be extended to include custom assertions, enhancing test readability and functionality.



Cypress Handout - 3

```
// Custom assertion to check if an element is visible
Cypress.Commands.add('isVisible', { prevSubject: true }, (subject) => {
  expect(subject).to.equal(document.activeElement);
});

// Usage in test
cy.get('[data-cy="input"]').isVisible();
```

Dynamic Selectors

Sometimes selectors need to be created during a test. For this type of conundrum, the following example can be used.

```
findElementOnTitle: (title: string) => {return `[data-cy="segment-${title}"]`};

// Usage in test
cy.get(findElementOnTitle('Search')).click();
```

Another option for dynamic selectors is starting with or ending with selectors. These can be useful when certain elements are generated inside the DOM.

For the **startingWith**, **^** indicates that this is only the first part of the selector.

```
startingWith: '[data-cy^="token-name-"]'
```

For the **endingWith**, **\$** indicates that this is the last part of the selector.

```
startingWith: '[data-cy$="-token"]'
```

While there are others just like these, use these options with caution. Any form of added complexity will reduce the maintainability and readability of your framework. Incomplete selectors require an extra degree of focus and attention in order to be readable.

Conclusion

Selectors are the backbone of both CSS styling and JavaScript DOM manipulation, but their importance is particularly pronounced in automated testing with frameworks like Cypress. Understanding how to effectively use selectors, traverse the DOM, and maintain selector stability is crucial for creating reliable and maintainable tests.

Cypress provides powerful tools for interacting with web elements, making it easier to write comprehensive tests that mirror user interactions. By following best practices such as using data attributes, avoiding overly specific selectors, and leveraging custom commands, you can ensure that your tests remain robust even as your application evolves.

In the ever-changing landscape of frontend development, mastering the art of selectors will empower you to create resilient and maintainable test suites, ultimately leading to more reliable and user-friendly web applications.

